

LLM-Assisted Annotation in VoxTool: Project Plan

Binoy Patel, September 2026

1. Goal

We want to add an LLM to VoxTool for one narrow job: reading the implant documentation PDF and filling in the lead definitions, so that whoever is annotating no longer retypes them by hand. Contact selection stays manual.

The workflow today is to open the PDF, read off each lead (its name, type and number of contacts), type all of that into the Define leads panel, and then check nothing was mistyped. With extraction in place the user would drop in the PDF, review a pre-filled table of leads next to the text it was taken from, correct anything wrong, and start annotating.

The main benefit is accuracy rather than speed. It saves maybe ten minutes of typing per patient, but more importantly a mistyped lead name or an incorrect contact count currently ends up in the saved `voxel_coordinates.json` with nothing to catch it. A second benefit is that documentation from different recording systems gets normalised into one schema.

2. What the LLM does and does not do

In scope: extracting lead definitions from documents, and normalising the different vendor formats into the schema VoxTool already uses.

Out of scope: deciding which bright voxel clusters in the CT are electrode contacts, placing or adjusting coordinates, or writing any coordinate the user did not click.

The reasoning is that telling contacts apart from surgical wire, staples and other metal in a CT is an image problem rather than a language one. Deep leads sit close to other hardware, and a language model asked to do that job will be confidently wrong some of the time. In a surgical planning tool a plausible but incorrect coordinate is worse than no suggestion at all, so keeping the human in charge of pointing at contacts is both more robust and easier to justify.

The model's output is therefore always shown alongside the source document, and always needs confirming before anything is written.

3. What already exists

VoxTool is a React frontend with a Flask backend, deployed two ways: a cloud instance on CloudFront, and an Electron desktop app (currently v1.0.7) that bundles the same frontend with a frozen copy of the backend.

Some of what was discussed as future work is already built and is not part of this project:

- Semi-automated contact tracing. The Interpolate function already handles the case where the user marks the first and last contact on a lead and the tool fills in everything between them, snapping each one to nearby bright voxels.
- Contact marking with intensity-based snapping, coordinate conversion between millimetres and voxels, the 2D slice and 3D electrode views, and saving or loading annotations as JSON or the legacy TXT format.
- A lead schema of name, type and dimensions. This is already the format the extraction step needs to produce, so the output format does not need designing.

This is why the extraction work is a small addition rather than a rewrite. The new surface area is one backend endpoint, one review screen, and a decision about where the model runs.

4. Questions to settle first

These are not engineering tasks, but the answers change the design, so they should be resolved before implementation starts.

1. Are the PDFs digital text or scanned images? Digital text is straightforward. Scans need OCR, which adds a dependency and a new way to fail, since LA1 misread as LAI would silently corrupt a lead name. This question roughly doubles or halves the scope.
2. What is the compliance path for patient documents? The PDFs contain identifiers, so sending them to a hosted API needs either a BAA covering that provider or de-identification before the file leaves the machine. The cloud deployment also has no authentication at present and is documented as demo-only. If there is no BAA route then the local model is not an optimisation but the only option, and hosted extraction has to be limited to de-identified documents.
3. How many distinct document formats are in use, and can we get examples of each? None of the work below can be built or evaluated without real samples.
4. Is this for Penn only, or for distribution to other centres? This decides whether the local or the hosted path is the priority.

5. Plan

Effort assumes part-time work alongside coursework rather than full-time. Each phase has an exit condition so progress is visible without guessing.

Phase 0: samples and ground truth

Collect five to ten de-identified example PDFs covering every recording system in use. Write out by hand the correct lead definitions for each one. That set becomes the reference used to evaluate every later decision. For each sample, note whether it is digital text or a scan, how many pages it has, and whether the lead information appears as a table or as prose.

Exit condition: the reference set exists with expected output for each sample, either in the repository or on a secure share if the documents cannot be committed.

No code is involved in this phase, and it is the real blocker. Everything after it is comparatively mechanical.

Phase 1: extraction schema and backend endpoint

Define the extraction schema formally, starting from the existing lead schema and adding a per-field marker for "not found" so that gaps are explicit rather than guessed at.

Add a `POST /api/extract/leads` endpoint to the Flask backend that takes a PDF and returns schema-valid JSON. Text extraction would use PyMuPDF, with OCR only if Phase 0 shows it is needed.

Implement it behind an interface with three interchangeable backends: a hosted API, a local open-weight model, and manual entry. The application must never depend on a model being available. Extraction pre-fills a form, and manual entry stays supported permanently.

Add validation that does not involve the model at all: contact counts must be positive integers, the lead dimensions must multiply out to the contact count, and lead names must match an expected pattern. Where the rules and the model disagree, raise it for the user rather than silently picking one.

Exit condition: the endpoint returns valid JSON for every Phase 0 sample using the hosted backend, and its accuracy against the reference set is measured and written down.

Phase 2: review and confirm screen

Add a step to the frontend where the user uploads a PDF, sees the extracted leads next to the source text they came from, edits any row, and confirms. Confirming populates the existing Define leads state, and nothing is written without that confirmation. Every field stays editable, and anything the model could not find is left blank and flagged rather than filled with a guess.

Exit condition: a complete run on a real scan, from PDF through confirmed leads and hand-marked contacts to saved coordinates, with no lead typing.

Phase 3: closing the pipeline

The preference from the planning meeting was to get the pipeline working end to end before adding anything else.

Support saving both to local disk, which already works, and to a configured Penn location. Implement the Penn upload as a setting plus a button rather than a natural-language instruction, for the reasons in section 7. Then confirm the whole path behaves the same in the desktop app and in the cloud build.

Exit condition: one documented end-to-end run on each of cloud and desktop.

Phase 4: local model evaluation and packaging

With a reference set and a working pipeline in place, the choice of model can be settled by measurement instead of assumption.

Benchmark small open-weight models against the Phase 0 reference set. The assumption that a small model will do because the task is simple is reasonable but untested, and clinical documents from several vendors are not necessarily clean. Use grammar-constrained decoding so the output is guaranteed to be valid against the schema, which removes malformed output as a category of failure and leaves only wrong values, which validation and human review already catch.

Two practical constraints are worth noting now. A 7 to 8 billion parameter model at 4-bit quantisation is around 4 to 5 GB, against current installers of 104 to 129 MB, and GitHub caps release assets at 2 GB per file. A bundled model therefore cannot ship through the existing desktop release workflow and would have to be downloaded on first run, or supplied by a local runtime such as Ollama. On the older laptops actually in use, a model that size runs at roughly 5 to 15 tokens per second on CPU, so a few hundred tokens of JSON takes 30 to 100 seconds. That is acceptable for a once-per-patient step provided it runs in the background with visible progress, in the same way the threshold cloud build already does.

Exit condition: a measured accuracy comparison of local against hosted on the same reference set, and a packaging decision justified by those numbers.

6. Hosted or local

Both are achievable, and the interface described in Phase 1 means the same schema and the same review screen serve either. The differences that matter:

Consideration	Hosted API	Local open-weight
Accuracy	Highest	To be measured in Phase 4
Cost	Cents per patient	None per use, one-off engineering
Patient data	Leaves the machine, needs a BAA or de-identification	Never leaves the machine
User setup	Needs an API account	None once installed
Speed	Seconds	30 to 100 seconds on older hardware

Cost should not drive this decision. A PDF is roughly 5,000 to 20,000 input tokens and under 1,000 output, so even a thousand patients is negligible. Compliance and distribution are what decide it.

7. One departure from the meeting

The meeting suggested replacing the save buttons with a prompt to the model, along the lines of asking it to save to Penn. I would recommend against this.

Saving is a repeated, well-defined action on patient data. Routing it through a language model makes it less predictable and introduces a way for a file to go somewhere nobody intended, in exchange for nothing a dropdown and a button do not already provide. The principle worth keeping is to use the model only where the input is genuinely unstructured, which here means the PDF and nothing else.

Worth revisiting if a conversational interface is wanted as a research contribution in its own right rather than as a usability improvement.

8. Not in the first version

Recorded so the boundary does not get relitigated part-way through: automatically identifying electrodes or contacts from the CT, any system that annotates without the user pointing at anything, a classifier separating electrode clusters from other metal, and natural-language control of file operations.

These are the obvious next direction, and the view in the meeting was that we are not there yet. The first version is a prerequisite for attempting them in any case, since it produces the confirmed lead definitions that an automated approach would need as input.

9. Risks

Risk	Consequence	Response
The PDFs turn out to be scans	Scope grows substantially	Settle it in Phase 0 before committing
No BAA route for patient documents	Hosted extraction unusable	The backend interface lets the local model carry the feature alone
Small local model not accurate enough	Packaging plan invalid	Reference set exists before the model is chosen, and hosted remains a fallback
Model output wrong but plausible	Corrupted coordinates	Confirmation is mandatory, source shown alongside, plus rule-based cross-checks
Model size and download	Cannot use the existing release pipeline	First-run download or an external runtime, decided in Phase 4

10. Relevance to the paper

The end-to-end pipeline is what makes this worth describing: an annotation tool where document understanding is automated, contact selection stays with the user, and the same workflow runs offline on a clinician's laptop or in a browser. The scoping decision in section 2, using the model where it is reliable and refusing it where it is not, is a defensible contribution on its own, and the Phase 0 reference set gives concrete numbers to report rather than claims.